

SOP-to-Executable: Fix History & Architecture

Sep 30, 2026 · @srini

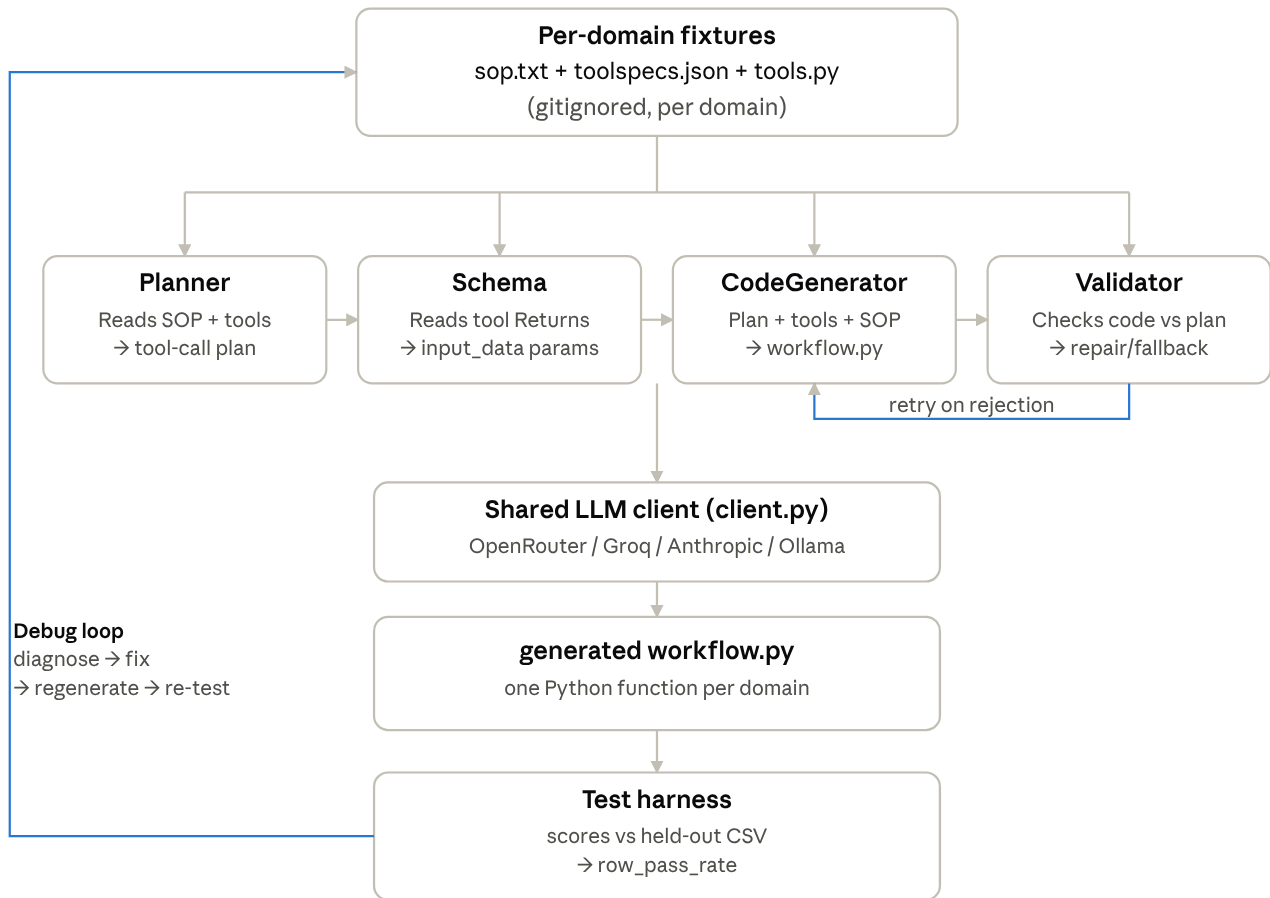
Full fix history across all 10 SOP-Bench domains and the shared LangGraph pipeline that converts each domain's plain-English SOP into an executable `workflow()` function, run against commit `1977b9c` and this session's uncommitted `eval_sops/` fixture state (2026-09-30).

Architecture

For every domain, four LangGraph agents turn a plain-English SOP plus a toolspec and a Python tool implementation into one generated `workflow()` function, which a test harness scores against a held-out CSV. All durable fixes land in the fixture layer at the top (SOP text, `tools.py`, `toolspecs.json`) or in the shared agent prompts — never as a one-off edit to generated code, since that doesn't survive the next regeneration.

Every domain runs the same five-stage pipeline, closed by a debug loop

Durable fixes land in fixtures (SOP text, tools.py, toolspecs.json), not in generated code



pipeline: fixtures → 4 LangGraph agents → generated workflow.py → test harness → debug loop back to fixtures

Current scores

All scores are full-dataset, zero-cost `python test_harness.py --domains <name> --test` re-scores of each domain's cached `workflow.py` — not a sample. The target bar was raised from 0.8 to 0.95 on 2026-09-29 once several domains proved 0.8 was reachable with pure fixture fixes.

Domain	Rows	Score	Bar (0.95)
aircraft_inspection_sop	112	1.00	met
patient_intake_sop	66	1.00	met

Domain	Rows	Score	Bar (0.95)
dangerous_goods_sop	274	1.00	met
warehouse_package_inspection_sop	150	1.00	met
video_annotation_sop	125	0.99	met (1 noise row)
email_intent_sop	186	0.92	not met (ceiling)
customer_service_sop	156	1.00	met
know_your_business_sop	90	0.80	not met
content_flagging_sop	168	0.00	not met
video_classification_sop	147	0.00	not met

8 of 10 domains are now at or above the 0.95 target, including `customer_service_sop`'s first-ever perfect score. The two 0.00 domains remain documented structural limitations (a data/ground-truth problem and a validator-vs-SOP conflict, neither fixable by prompt/toolspec changes). `know_your_business_sop` sits at a documented noise ceiling. `email_intent_sop` sits just under the bar at its own documented ceiling — a duplicate-key data collision that needs a new tool capability, not a prompt fix.

Cross-cutting pipeline fixes

Fixes below live in the six tracked pipeline files (in git) and affect every domain, not one. Per-domain fixes (SOP text, `tools.py`, `toolspecs.json`) are gitignored fixture data and listed in the next section.

`client.py`

- Added `anthropic`, `ollama`, and `openrouter` as providers alongside the original `groq`. Both `ollama` and `openrouter` were initially broken: they reused Groq's SDK as a generic HTTP client, but that SDK hardcodes its request path (`/openai/v1/chat/completions`), which misroutes against a different `base_url`. Fixed by switching both to the real `openai` package's client. `openrouter` is now the default provider.
- Added `timeout=120.0` to all three provider constructors. Previously a connection killed mid-flight (e.g. laptop sleep) hung forever — observed once as 9+ hours at 0% CPU with no output. Documented as **not fully sufficient**: a retry's own connection can

hit a second sleep cycle and hang again past the 120s timeout, since a torn-down network interface after wake doesn't always surface as a clean socket error on an already-established connection. Mitigation is operational (`caffeinate -i` on long runs), not a further code fix.

- Added cumulative token-usage tracking (`get_usage()` / `reset_usage()`), which `test_harness.py` 's per-domain token reporting and this session's budget tracking both depend on.

`planner_agent.py`, `schema_agent.py`, `codegeneration_agent.py`,
`validation_agent.py` (shared fixes)

- **Blank/malformed LLM response handling made retryable.** All four agents previously raised a custom, unretried exception (e.g. `PlannerAgentError`) on a blank or malformed response, which propagated straight out and killed the entire pipeline run on the first occurrence. Root cause: OpenRouter's reasoning model (`openai/gpt-oss-120b`) non-deterministically spends part of its token budget on a hidden `reasoning` field before emitting visible `content`, so the identical prompt can return real content on one call and nothing on the next. Changed to `RuntimeError`, which the existing retry loop already catches.
- **Guardrail-rejection retries widened.** In `schema_agent.py` and `codegeneration_agent.py`, only the raw LLM call was being retried — a rejection at the parse or output-guardrail step afterward still ended the run on the first attempt. Restructured `__call__` so the LLM call, extraction, and guardrail checks retry together as one loop.
- `json.loads(raw, strict=False)` parsing generalized across all four agents, after recurring "Unterminated string" failures traced to literal (unescaped) newlines inside JSON string fields.
- `max_tokens` raised multiple times across all four agents as domains grew (from a 2000 default up to 8000 for the largest domains/providers) — driven mostly by Groq's 8000 TPM ceiling interacting with `customer_service_sop` 's 10-tool size, and later by OpenRouter's reasoning-model overhead.

`schema_agent.py` (own fixes)

- **`_compute_parameter_checklist()` added.** Previously the LLM was asked to recall every tool's parameters from prose — this degraded badly at scale (found at 26 tools in `video_annotation_sop`: invented parameter names that don't exist, and missed real

ones). Replaced with a deterministic, code-computed checklist of every real parameter and return-field name, turning "recall N names from prose" into "sort each name in this precomputed list," which doesn't degrade with tool count.

- **Parameter-naming rule flipped from forced snake_case to exact-spelling preservation.** `input_data` keys are raw CSV column headers; normalizing a name that no longer matches the real header caused a silent `KeyError` at runtime.
- **Parameter checklist split into required vs. optional-with-default (2026-10-01).** The checklist forced every tool parameter — required or not — into a binary classification ("covered by an earlier return" or "a base-level input"), no third option for "optional, has its own documented default, just omit it." An optional flag with a `default: false` and no mention anywhere in its SOP got wrongly promoted to a required `input_schema` field with no corresponding CSV column, crashing every row in `email_intent_sop` identically with the same `KeyError`. Fixed by splitting on the `required / default` metadata the toolspec already provides (via `tools_helper.py`, below) — domain-general, not a one-off.

tools_helper.py

- **Nested Returns fields recursively extracted and rendered (2026-10-01) — the single most consequential fix of the project.** `load_tools_from_toolspec_json()` only ever extracted the TOP level of a tool's `outputSchema.properties`. For a Returns field documented as a nested object (e.g. an "authentication records" object whose own `properties` named `login_status / account_recovery_status` with full enum values), only that field's own `type / description` were captured — its nested field names were silently dropped before ever reaching a prompt. Every agent (planner, schema, codegen, validator), for every domain with this shape, saw only `Returns: {'authentication records': {'type': 'object', 'description': "...exactly these keys..."}}` — the real field names were never visible anywhere in the pipeline. This had been misdiagnosed for 4+ regeneration attempts across sessions as "a confirmed model sampling habit"; tracing the literal rendered prompt text (not just re-reading the source toolspec, which looks fully correct in isolation) found the real cause. Fixed with a recursive `_extract_schema_fields()` plus a `_render_returns()` that shows a nested field's FULL bracket access chain relative to the tool's response (e.g. `['authentication records']['login_status']`) — a first attempt using indentation alone was insufficient: the model correctly used the real field name but still tried a flat access and got a `KeyError`, since indentation shows nesting exists but not which keys are required to reach it.

codegeneration_agent.py (own fixes)

The longest-running fix target in the project — grew from an initial handful of rules to 26 numbered "CRITICAL REQUIREMENTS," each added after a specific, diagnosed failure.

Highlights:

- **Retry feedback wasn't being read.** The orchestrator wrote `retry_feedback` to state, but the code generator never read it — every retry regenerated from scratch with no memory of what failed. Fixed to embed both the prior feedback and the prior generated code into the retry prompt.
- **Return-value truncation.** Generated code was computing every SOP-required output field correctly, then `return` ing only the last step's bare value, silently discarding the rest (rule: combine every step's result, not just the last one).
- **Decision logic defaulting to shortcuts instead of named conditions** — the recurring pattern (rules 14, 17, 18, 24, 25) of a generated status field defaulting to a constant, or to `bool(some_dict)`, instead of checking the specific named field the SOP and the tool's documented Returns actually specify. This pattern is the confirmed, still-open root cause of `customer_service_sop`'s plateau (see per-domain section).
- **Output key/value shape rules** — flat canonical keys instead of prose-derived nesting, SOP's own compound terminology instead of bare abbreviations, never invent a hardcoded literal when a real source exists, never substitute a "simplifying assumption" for a documented check, cast `input_data` string values before numeric/boolean comparison.
- **Early-termination checklist (rule 23)** — scan the whole SOP for early-stop phrasing before writing code; a termination condition almost always means stop calling tools entirely, not skip-one-step-and-continue.
- `max_tokens` raised 2000 → 8000 over the project, plus a `MAX_PREVIOUS_CODE_CHARS` cutoff (drop the previous attempt's code from retry prompts past 4000 characters) to stay under Groq's TPM ceiling on large domains.

validation_agent.py (own fixes)

- **Over-escaped-newline repair.** The model would write the literal two characters `\n` (not a real newline) inside `corrected_code`, which `json.loads()` decodes correctly per spec, so a 169-line file was seen as one line and rejected by a minimum-line-count guardrail. Fixed by detecting this pattern and un-escaping before the check, rather than rejecting.

- **tools_formatted** added to the validator's own input and prompt — it was the only one of the four agents missing this. Without it, the validator was flagging "parameter isn't part of the documented signature" by comparing against the planner's `api_plan` (which never carries a parameter list at all), which drove the code generator to progressively strip a genuinely required parameter across retries until the call broke.
- **_resolve_final_code()** fallback gap fixed. When `corrected_code` failed its own post-checks (syntax, required imports), the failure previously propagated straight out and ended the pipeline run — even though the adjacent "no `corrected_code` provided" branch already had a working fallback to `generated_code`. Fixed by wrapping the check in a `try/except` that falls back to `generated_code` (which already passed the code generator's own guardrails).

test_harness.py (own fixes)

- **Critical scoring bug, retroactively affected multiple domains' recorded history.** The scoring loop's own comment claimed to check nested dict values, but only built its comparison set one level deep. A second bug: a ground-truth list stored as a bracketed string (e.g. `"['Wrong Item']"`) was compared by exact string equality against flattened actual values, which could never match; and `"0.0"` vs `"0"` never matched as strings. All three fixed (`flatten_values()` recursion, an `ast.literal_eval`-based list comparison, and numeric-equivalence matching). Several domains' already-correct generated code had been scored as wrong before this fix — any score recorded before 2026-09-28 against a domain with a list-shaped or numeric output field should be treated as a potential undercount.
- **--test flag added:** re-scores whatever `workflow.py` is already cached for a domain against the full test set, at zero API cost. This is the mechanism behind every "free" re-verification in this document.
- **MAX_TEST_ROWS** raised from a 3-10 row debugging sample to 10000 — every score in this document is now a full-dataset score, not a sample.
- **verify_toolspec_matches_manager()** added, later extended with AST-based stub-method detection (flags a tool method whose body is just a docstring + `pass`) and exemption of optional (defaulted) parameters from the missing-parameter check. Runs as a non-fatal warning before any LLM call, catching toolspec/implementation mismatches that previously surfaced only as a confusing runtime error several LLM calls deep.

Per-domain fix history

Each domain's SOP text, `tools.py`, and `toolspecs.json` live under `eval_sops/<domain>/` and are gitignored (not committed) — these fixes are documented here and in `sop_autoresearch.md` / `CURRENT_SCORES.md`, not visible in `git log`.

aircraft_inspection_sop — 1.00 / 112 rows, bar met

- **SOP fixes:** none documented.
- **Tools.py fixes:** none documented.
- **Toolspec fixes:** none documented.
- **LangGraph agent fixes:** none — this domain is never named in any fix commit or research-log item.
- **Auto research agent fix:** none specific to this domain.
- **Other fixes:** none.
- **Gap in the record:** as of 2026-09-27 the research log states this domain "does not exist as an actual folder yet." Two days later it's scored at 1.00/112 with no onboarding narrative, fix log, or debugging session recorded anywhere in either tracking file. Flagged honestly rather than invented — it may simply have needed no fixes once onboarded, but that isn't documented.

patient_intake_sop — 1.00 / 66 rows, bar met

- **SOP fixes:** none documented.
- **Tools.py fixes:** none documented directly — the bug below was a documentation gap, not a code bug.
- **Toolspec fixes:** `toolspecs.json` had no `outputSchema` entries at all, which (combined with a codegen default bug) caused type-confusion crashes.
- **LangGraph agent fixes discovered via this domain:**
 - `codegeneration_agent.py`: fixed `return` only passing through the last step's result instead of combining all 6 required output fields.
 - `codegeneration_agent.py`: flipped the default assumption for an undocumented tool return from "assume dict" to "assume scalar unless Returns names fields" — root cause of repeated `'str' object has no attribute 'get'` crashes, since every tool here actually returns a bare string.

- **Auto research agent fix:** `test_harness.py`'s nested-value scoring bug (see Cross-cutting section) was confirmed via this domain — `--test` showed it jump to a genuine 1.00 once the harness could see nested values correctly.
- **Other fixes:** none.
- **Status:** stable at 1.00, no remaining known issues.

`email_intent_sop` — 0.92 / 186 rows, just under bar (at documented ceiling)

- **SOP fixes:** added an implementation note flipping which category is the structural fallback — generated code was gating "generic question about a listing" on a literal `"?"` character and using "unable to decide" as the catch-all else-branch, exactly backwards from the SOP's own existing guidance that (e) should be rare. Clarified that (d) is the default for any listing-related email not matching (a)/(b)/(c), regardless of punctuation, and (e) is reserved for content that isn't recognizably about the listing at all.
- **Tools.py fixes:** a `marketplace_id` regex required 2 letters + 3 digits, but 100% of the dataset uses the bare 2-letter form with no digits — relaxed the digits to optional.
- **Toolspec fixes:** fixed a filename mismatch and added missing `outputSchema` entries for all 5 tools; `get_product_price` required a `marketplace_id` parameter the toolspec never declared.
- **LangGraph agent fixes discovered via this domain:** a second genuine shared-pipeline bug, found via a real regeneration (not `--test`, which never exercises prompt-building code at all) — `schema_agent.py`'s parameter checklist lumped every tool parameter, required AND optional, into one list the prompt forces into a binary classification ("covered by an earlier return" or "a base-level input"), with no third option for "optional, has its own default, just omit it." An optional `include_history` flag (default `false`, never mentioned in the SOP) got wrongly promoted to a required input field with no corresponding CSV column, crashing every single row identically. Domain-general, not unique to this SOP's wording — fixed by splitting the checklist using `required / default` metadata the toolspec already provides, verified against all 10 domains' real toolspecs before the next regeneration.
- **Auto research agent fix:** `verify_toolspec_matches_manager()` (Cross-cutting section) was built partly in response to this domain's undeclared-parameter bug, in an earlier session.
- **Other fixes:** none.

- **Remaining known issue (same ceiling as before, not a regression):** a duplicate `product_id + marketplace_id` key collision across two different emails with no disambiguating key — a genuine data/tool-signature gap needing a new tool parameter, not a prompt fix. A fresh regeneration during this session's full-pipeline pass fixed the crash and the classification-fallback bug above, landing back at this exact same pre-existing ceiling (~0.92–0.95 depending on sample) rather than a new problem.

dangerous_goods_sop — 1.00 / 274 rows, bar met

- **SOP fixes:**
 - Added an explicit "0/missing is not a validation failure here, only negative/>5 is" caveat, resolving a self-contradiction where an earlier section's hard validation gate ran before a later section's correct imputation logic ever got a chance to run — this alone had been crashing 28/274 rows.
 - Changed "if more than two component scores are missing" to "two or more" — ground truth showed exactly 2 missing scores already forces `Unable to Decide` in all 5 such rows.
 - Added exact numeric hazard-score-to-class thresholds (derived from groupby min/max on clean rows) — the SOP previously gave no numeric boundaries, so codegen guessed a boundary that misclassified a borderline ground-truth row.
- **Tools.py fixes:** all four `calculate_*_score` methods crashed with "cannot convert float NaN to integer" on a genuinely blank CSV cell — fixed to treat a NaN cell as 0/missing instead of crashing.
- **Toolspec fixes:** `assessmentFormId` was declared in the toolspec but never implemented on the manager (caught by `verify_toolspec_matches_manager()`).
- **LangGraph agent fixes discovered via this domain:** the planner's omit-tool-less-steps rewrite; validator `strict=False` JSON parsing; codegen output-terminology and flat-key rules (a sample had correctly computed `hazard_score / hazard_class` but nested them under prose-derived keys instead of flat ones, scored as a total miss).
- **Auto research agent fix:** this was one of the two domains (with `patient_intake_sop`) that confirmed the nested-value scoring bug fix via `--test`; the list/numeric scoring bug fix (Cross-cutting section) also retroactively affects this domain's output format.
- **Other fixes:** none.
- **Status:** reached 1.00/274 via two regenerations (53,160 tokens) after the SOP/tools.py fixes above; confirmed via `--test`, no remaining known issues.

customer_service_sop — 1.00 / 156 rows, bar met (first-ever perfect score)

- **SOP fixes:** three, found only once real regeneration (not just `--test`) was tried: (1) 5.5 Troubleshooting said "re-execute diagnostics" for post-troubleshooting metrics, but the diagnostics tool is a static lookup that returns identical numbers on every call — confirmed by direct instrumentation. Added a sentence naming `executeTroubleshooting`'s own response as the real source. (2) The output field `metrics_improved_post_troubleshooting` was only ever named in the Output JSON example, never tied to the "classify as fixed" threshold rule two sentences earlier — generated code computed the right check into an unused variable and a wrong relative-decrease formula into the actual output field. Added an explicit bridging sentence with a counterexample (a 500ms→150ms drop still counts as "not improved" since 150 exceeds the 100ms threshold).
- **Tools.py fixes:** none needed this round.
- **Toolspec fixes:** `account_suspension_status`'s enum only listed `["ACTIVE", "SUSPENDED"]`, never documenting that an empty string means "no suspension on record" — the common case (132/156 rows). The model reasonably treated anything `≠ "ACTIVE"` as a failure, terminating rows that should have proceeded. Fixed the enum and description to name all three states explicitly.
- **LangGraph agent fixes discovered via this domain:** the real breakthrough was a shared-pipeline bug, not an agent-prompt one — `tools_helper.py` was silently flattening nested toolspec Returns fields (see Cross-cutting section), hiding `login_status` / `account_recovery_status` from every agent the entire project. The only reason any prior attempt ever produced those field names was a prompt engineer's worked example in codegen rule 24 leaking them as an aside — not something the model derived from its own tool documentation, which is exactly why it was unreliable.
- **Auto research agent fix:** this domain alone needed a toolspec-rendering fix, a toolspec-enum completeness fix, and two SOP-prose-to-output-field bridging fixes, across 8 total regeneration attempts, before reaching 1.00. Each of the three bugs found in the final regeneration pass was isolated by wrapping every manager method with a print/trace instrumentation layer and running one real failing row through it end-to-end — necessary because two different bugs produced the identical generic error "No record found for the provided parameters," which could have come from any of five different tool calls.
- **Other fixes:** none.

- **Status:** what was documented for 4 regenerations as "a confirmed model sampling habit" turned out to be a tooling bug the whole time — the model was never shown the real field names at all, on any domain with a nested-object tool Returns shape. Once visible, a real regeneration pass surfaced (and fixed) three more narrow, well-isolated bugs, reaching **1.00/156**.

video_annotation_sop — 0.99 / 125 rows, bar effectively met

- **SOP fixes:** none for the fix that took this domain from 0.83 to 0.99 — purely a tools.py fix (below).
- **Tools.py fixes:** an AST-based stub check found 20 of 26 tool methods (77%) were bare `pass` stubs when this domain was onboarded. Later, three more tools were found to be *functionally* stubs — `validateSceneContext`, `calibrateCameraSensors`, `executeSegmentation` each returned `is_valid: True` unconditionally once a matching CSV row was found, never checking the actual categorical value against the SOP's real constraint. Fixed to check `scene_type` against the "urban setting" requirement, `camera_position` against "front-camera positioning," and `segmentation_type == 'instance'` respectively (ground truth: `panoptic / semantic` segmentation types are 100% failing, 9/9 and 17/17 rows). Also fixed a crash where a legitimate empty `predicted_object` (a real "no object detected" outcome) was treated identically to a missing parameter and raised instead of flowing through as a failed row — same bug existed in two other tools and was fixed in all three.
- **Toolspec fixes:** none specifically named beyond general onboarding (26 tools, this repo's tool-count ceiling).
- **LangGraph agent fixes discovered via this domain:** `schema_agent.py`'s `_compute_parameter_checklist()` rewrite was built specifically because this domain's 26-tool scale broke LLM-based parameter recall; `test_harness.py`'s AST-based stub detection was built specifically in response to this domain's 20/26 bare-`pass` stubs.
- **Auto research agent fix:** documented in the research log as the domain that proved "a tool returning a real-looking value that never actually depends on the field it claims to validate" is a distinct, harder-to-detect bug class than a bare stub.
- **Other fixes:** none.
- **Status:** fixed entirely at zero token cost (pure tools.py fixes, no regeneration needed — `workflow.py` was already correctly ANDing every tool's `is_valid` flag together). One residual row (`vid_00133`) has every metric clear of every threshold with no discriminating feature found — treated as ground-truth noise, not pursued further.

know_your_business_sop — 0.80 / 90 rows, below bar

- **SOP fixes:** section 5.6.2 rewritten — previously checked 7 escalation triggers before checking whether any UBO's sanctions status was still "Pending." Ground truth showed the reverse on the 50 overlap rows (68% were "awaiting information" even with another UBO already "Matched"). Rewrote to check "any UBO still Pending" first, unconditionally.
- **Tools.py fixes:** `performSanctionsCheck()` only ever looked up sanctions/PEP status for the *first* UBO in the list, silently dropping every other UBO — and in this dataset the still-Pending UBO was consistently not in position 0. Fixed to iterate every UBO. This fix alone (free re-test, no regeneration) moved the score from 0.61 to 0.80, after the SOP fix alone had only moved it from 0.60 to 0.61.
- **Toolspec fixes:** added missing `outputSchema` entries, and documented that `sanction_check_status` uses a Clear/Pending/Matched vocabulary while `pep_status` uses Yes/No — previously undocumented, which had caused a copy-pasted comparison bug.
- **LangGraph agent fixes discovered via this domain:** four separate `codegeneration_agent.py` rules were diagnosed directly via this domain's escalation-logic bugs — decision logic must follow named conditions (not a risk-score shortcut), read the whole SOP for scattered trigger conditions rather than just the section titled as the decision section, never check a key absent from a tool's documented Returns, and don't reuse one field's comparison logic for a different field with a different vocabulary.
- **Auto research agent fix:** an exhaustive single- and paired-feature search across every visible column (documented in the research log) confirmed the escalation-order fix was right, and separately confirmed no feature discriminates the remaining noise rows (below).
- **Other fixes:** none.
- **Remaining known issues (18/90 rows, investigated and not pursued further):** 16 rows are "escalate despite a different UBO still Pending" cases where an exhaustive feature search found zero features that separate them from the 34 "awaiting information" rows with the same trigger pattern — the signature of noise on top of a base rate, matching the SOP's own warning that its risk score "may not accurately capture all the relevant information." 2 rows (`biz_048` , `biz_008`) are a different, identified issue: each shares an identical registration number/license/tax ID/bank account with a *different* `business_id` elsewhere in the same CSV — a real identity-fraud signal, but catching it

needs a cross-row lookup tool that doesn't exist and that the per-row `workflow()` contract can't support on its own. Flagged for a future decision rather than built, since it's a new capability the SOP implies but never specifies as an actual tool.

warehouse_package_inspection_sop — 1.00 / 150 rows, bar met

- **SOP fixes:** added a section describing that `resolution_status` depends on the `chargeable` input column when `problem_type` is empty, and documented `chargeable` itself as an input field (it wasn't previously documented anywhere the pipeline could see it).
- **Tools.py fixes:**
 - `calculateChargeback()` only special-cased 2 of 5 "quantity discrepancy" problem labels, silently charging \$0 for the other 3, and fabricated a 10% "handling fee" for Wrong Warehouse with no basis in the SOP. Fixed to charge the quantity-discrepancy amount once per row whenever any of the 5 labels is present (not per-label), plus a damage charge if Vendor Damaged is present, additively, \$0 for Wrong Warehouse. Also fixed `chargeable` from `total_charge > 0` to `total_charge != 0`, since an Overage Quantity row's chargeback is a legitimate negative credit.
 - `generateProblemReport()` raised an error on any negative charge amount, crashing every overage-only row even after the fix above computed the correct negative value — removed, since a negative chargeback is valid.
 - `updateResolutionStatus` was hardcoded to always return "Resolved" when `problem_type` was empty — implemented the actual branch ("Returned to Vendor" vs "Resolved") based on the `chargeable` input.
- **Toolspec fixes:** added `chargeable` as a required parameter to `updateResolutionStatus`'s input schema.
- **LangGraph agent fixes discovered via this domain:** the `test_harness.py` list/numeric scoring bug fix (Cross-cutting section) was discovered while onboarding this domain, since its `problem_type` field is a list.
- **Auto research agent fix:** none beyond the scoring-bug discovery above.
- **Other fixes:** none.
- **Status:** reached 1.00/150 across two passes — 0.60→0.95 at zero token cost (pure tools.py fixes), then 0.95→1.00 via one regeneration (24,142 tokens) to thread the `chargeable` fix through.

content_flagging_sop — 0.00 / 168 rows, below bar (structural)

- **SOP fixes:** none — not fixable by prompt or toolspec changes without inventing new business logic.
- **Tools.py fixes:** none applied; the root cause is not a tools.py bug.
- **Toolspec fixes:** none.
- **LangGraph agent fixes:** none — this is a documented dead end, not a source of any agent-level fix.
- **Auto research agent fix:** the research log documents the diagnostic method — reverse-engineering what `device_consistency_score` would need to be to reproduce the recorded `user_trust_score` ground truth, which comes out well outside its own documented 0–1 range and varies for identical device/OS/browser combinations.
- **Other fixes:** none.
- **Status:** `user_trust_score` 's formula is mathematically disconnected from its own ground truth. Re-verified twice (156 then 168 rows as the dataset grew) with no new evidence either time — not re-investigated further per the project's own stop-rule for structural limitations.

video_classification_sop — 0.00 / 147 rows, below bar (structural)

- **SOP fixes:** three escalating rounds of increasingly forceful wording, up to stating the conditional skip as a "MANDATORY, NON-NEGOTIABLE CONDITION" — none of the three stuck.
- **Tools.py fixes:** none — `moderator_id` is genuinely blank for non-escalated rows in the source data, which is the correct state, not a bug.
- **Toolspec fixes:** none.
- **LangGraph agent fixes:** none applied yet — this is the domain's actual root cause and the fix is still pending (see status).
- **Auto research agent fix:** the research log documents the root-cause finding across 7 regeneration attempts — `validation_agent.py`'s own "every API-plan step must be implemented" bias overrides an explicit, SOP-documented conditional skip, no matter how forcefully the SOP states the condition. The validator's own feedback independently re-asserts "steps are missing, call every listed tool" each retry, and the code generator capitulates to the validator over the SOP — described as "a genuine tug-of-war between two agents reading the same SOP differently," not a one-sided prompt gap.
- **Other fixes:** none.

- **Status:** structurally blocked by a `validation_agent.py` behavior, not a fixture problem — the only domain in this project where the identified fix is scoped to `validation_agent.py` itself rather than SOP text, `tools.py`, or `toolspecs.json`. The most recent agent-prompt changes targeted `customer_service_sop`'s bug, not this one; this domain's specific fix (teaching the validator that a step legitimately absent because its trigger condition wasn't met is not the same defect as an oversight) has not yet been attempted.

Chronology

1. **Initial uploads and early planner work.** Baseline repo with a single `groq` provider, no retry logic, no token tracking.
2. **2026-09-18 — Autoresearch loop established.** `sop_autoresearch.md` and the experiment-log discipline set up; first real baseline run against the original 3 domains used ~199,000 of a 200,000 daily token quota on Groq.
3. **2026-09-19-20 — First fix wave.** Token usage tracking added; the `--test` zero-cost re-scoring flag added; the critical nested-value scoring bug fixed (several domains' real scores had been hidden by this); rapid-cycle fixes against `patient_intake_sop`, `dangerous_goods_sop`, `know_your_business_sop`.
4. **2026-09-23-24 — Scale-driven fixes.** `customer_service_sop` (10 tools) onboarded, triggering a dedicated debugging arc dominated by Groq's 8000 TPM ceiling — `max_tokens` tuning, prompt condensation, dropping oversized retry context. `email_intent_sop` onboarded in parallel; the toolspec-vs-manager verification check added after two domains separately cost real token spend on the same root cause (an undeclared-but-required tool parameter).
5. **2026-09-26 — OpenRouter added** as a client provider (initially broken, properly fixed in the next session).
6. **2026-09-27 — Triage pass.** `video_annotation_sop` onboarded as a 6th domain; a "per-domain triage" process documented, noting 4 more domains existed only in a dataset manifest, not as real folders yet.
7. **2026-09-28 — The major consolidated fix session.** Fixed the broken OpenRouter/Ollama routing; added client timeouts; rewrote the schema agent's parameter-detection logic around a deterministic checklist; fixed the two critical test-harness scoring bugs (list matching and numeric formatting); made all four agents' failure paths retryable instead of fatal. `CURRENT_SCORES.md` created as the first verified score table.

8. **2026-09-29, first debug-loop pass — target: 0.8.** The 4 newly-onboarded domains brought online; `warehouse_package_inspection_sop` (0.60→0.95) and `know_your_business_sop` (0.60→0.80) fixed; `content_flagging_sop` / `video_classification_sop` confirmed as structural dead ends at 0.00.
9. **2026-09-29, second debug-loop pass — target raised to 0.95.** `dangerous_goods_sop` (0.90→1.00), `warehouse_package_inspection_sop` (0.95→1.00), and `video_annotation_sop` (0.83→0.99) fixed; `customer_service_sop` regenerated once more and reproduced the same confirmed bug for a 4th time; the other three below-bar domains re-verified unchanged. This is the current state of the project.

Known gaps in this record

Flagged explicitly rather than smoothed over, since this document is meant to be reviewed in detail:

- **aircraft_inspection_sop has no documented debugging history.** It's listed as "doesn't exist as a folder yet" on 2026-09-27 and scored at 1.00/112 by 2026-09-29, with nothing in between recorded in either tracking file.
- **Three domains (`email_intent_sop`, `dangerous_goods_sop`, `warehouse_package_inspection_sop`) have a retroactive-correction note in the research log** stating the list/numeric test-harness scoring bug fix alone moved their scores to 1.00 on unchanged, already-cached code — but the score tables recorded immediately afterward show lower figures for all three (0.95, 0.90, and a separate 0.60 starting point, respectively), with 1.00 only reached later through additional real fixes. The exact relationship between the "free" scoring-bug correction and the subsequent fixes isn't reconciled in the source documents — possibly the retroactive-correction note describes a different `workflow.py` snapshot than the one scored right after.
- **customer_service_sop's bug status is ambiguous at the margin.** The latest agent-prompt rules (24–25) were written specifically to target its root cause, but a regeneration tested around the same time still reproduced the bug. It isn't clear from the record whether that test ran before or after the new rules took effect.

Orchestrator escalation (implemented 2026-10-01)

This started as a design discussion and was implemented the same day. The graph (`agent_pipeline.py`) was linear with exactly one retry edge — `orchestrator` → `generator` — so planner and schema each ran once per invocation, and every retry regenerated code from the same plan and schema even when those were the actual problem. Now the orchestrator detects when a validation issue *persists unchanged* across a codegen retry, classifies its shape (plan-shaped vs schema-shaped vs code-shaped) via `_choose_retry_target()` , and routes to planner or schema instead of just generator again — gated by its own small retry budget (`plan_retries` / `schema_retries`) so it can't loop indefinitely. A new `plan_diff_checker.py` node runs a cheap mechanical check (did the new plan actually change?) after a planner-escalation before spending tokens on a full schema/codegen/validator re-run, failing fast if the replan came back identical.

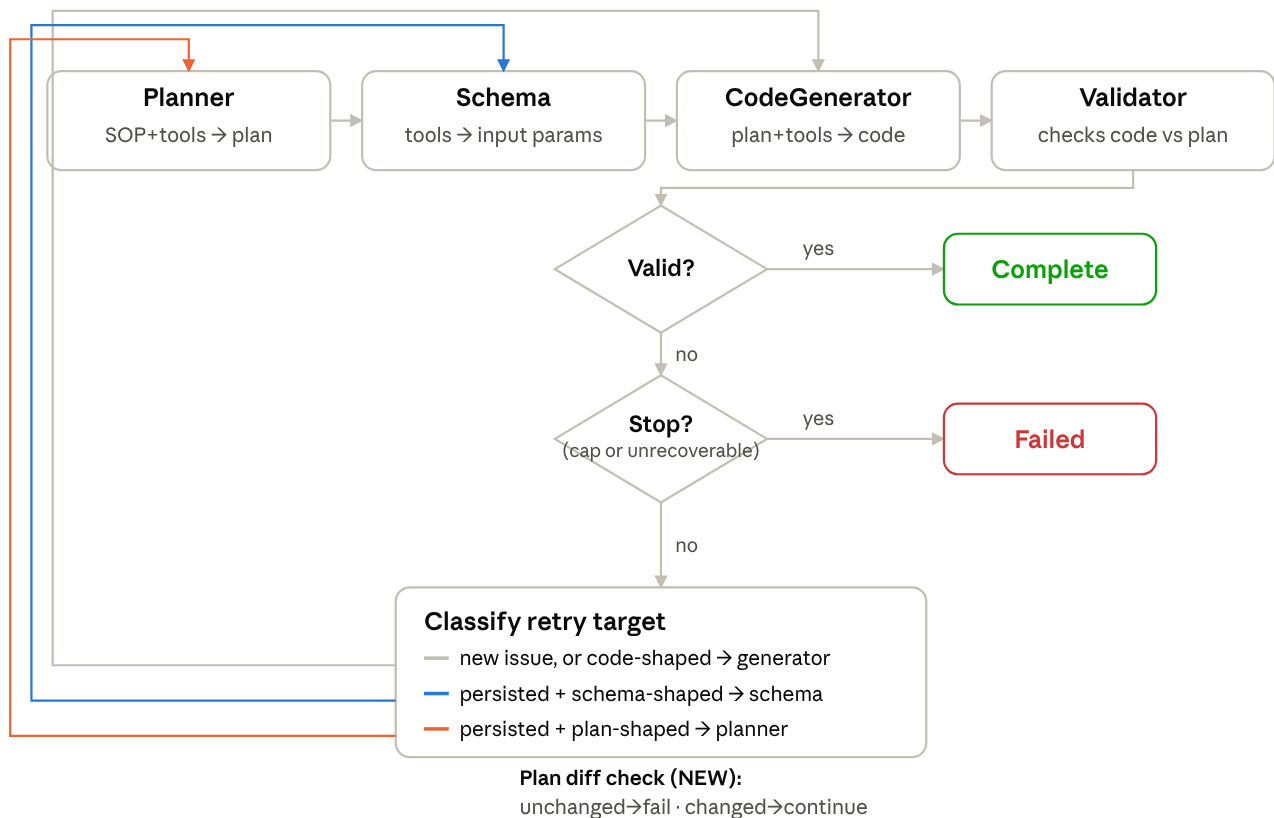
Orchestrator classifies a persistent issue by shape before escalating

Implemented 2026-10-01. Gray = pre-existing path. Color = new escalation.

— retry → generator (existing)

— escalate → schema (NEW)

— escalate → planner (NEW)



implemented: orchestrator escalates to schema or planner on a persistent issue, gated by a mechanical plan-diff check

Implemented, verified via direct unit tests of the escalation logic and the plan-diff gate (all three target branches and all three gate outcomes confirmed correct), graph-compile verification, and zero-regression `--test` checks across all 10 domains. Not yet exercised live in a real regeneration where it actually fires — the bugs found this session (e.g. `customer_service_sop`'s `bool-of-dict` pattern) classify as code-shaped, so the existing `generator-retry` path handled them; the `schema/planner` escalation branches are implemented and tested in isolation but haven't yet been the deciding factor in a real fix.